



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2213 - LÓGICA PARA CIENCIAS DE LA COMPUTACIÓN
PROFESOR: MIGUEL ROMERO
AYUDANTES: ALEXANDER PINTO Y ALEJANDRA SCHILD

Ayudantía 10

Problema 1. Demuestre que, si $P = NP$, entonces existe una máquina determinista de tiempo polinomial que, dado un grafo G , calcula y entrega de salida un camino hamiltoniano de G en caso de que exista alguno, o bien entrega de salida el símbolo $\perp$ en caso contrario.

Solución: Consideremos el lenguaje PREFIXHAM de los pares $\langle G, \sigma \rangle$, donde $G = (V, E)$ es un grafo y $\sigma \in V^k$ es una tupla (posiblemente vacía) de k vértices distintos de G , tales que existe un camino hamiltoniano en G que tiene a σ de prefijo, es decir, que coincide con σ en los primeros k vértices. En particular, si σ es una tupla vacía, simplemente requerimos que G tenga un camino hamiltoniano. Notemos que PREFIXHAM $\in$ NP, pues en tiempo polinomial podemos adivinar un orden total de los vértices de V y verificar si este es un camino hamiltoniano que además tiene a σ de prefijo. Como estamos asumiendo que $NP = P$, tenemos una máquina determinista $\mathcal{A}$ que decide PREFIXHAM en tiempo polinomial.

Ahora consideremos el siguiente algoritmo:

Entrada: Un grafo $G = (V, E)$;

- 1: inicializar una lista vacía **prefijo**;
- 2: inicializar un conjunto **nodos_restantes** con todos los vértices de V ;
- 3: **if** $\mathcal{A}(G, \text{prefijo}) = 0$ **then**
- 4: **retornar** $\perp$;
- 5: **end if**
- 6: **while** **nodos_restantes** $\neq \emptyset$ **do**
- 7: **for** $v \in \text{nodos_restantes}$ **do**
- 8: **if** $\mathcal{A}(G, \text{prefijo} + [v]) = 1$ **then**
- 9: concatenar v al final de **prefijo**;
- 10: quitar v de **nodos_restantes**;
- 11: volver a la línea 6;
- 12: **end if**
- 13: **end for**
- 14: **end while**
- 15: **retornar** **prefijo**.

Si el algoritmo no termina en la línea 4, significa que G sí tiene un camino hamiltoniano. En ese caso, notemos que solo agregamos un vértice nuevo v a **prefijo** si ya revisamos que existe un camino hamiltoniano que respeto el prefijo e inmediatamente después pasa a v . Por lo tanto, tenemos la garantía que en todas las iteraciones del bucle ubicado entre las líneas 7 y 13 extenderemos la lista **prefijo** de manera válida. Luego lo que retorna el algoritmo es efectivamente un camino hamiltoniano. Hacemos $O(|V|^2)$ llamadas a la máquina $\mathcal{A}$, que funciona en tiempo polinomial, por lo que el algoritmo anterior también funcionará en tiempo polinomial. Esto completa la demostración.

□

Problema 2. Considere el siguiente problema de decisión:

Nombre: INTEGERPROGRAMMING

Entrada: Una matriz $A \in \mathbb{Z}^{n \times m}$ y un vector $\vec{b} \in \mathbb{Z}^{n \times 1}$.

Decisión: ¿Existe un vector $\vec{c} \in \mathbb{Z}^{m \times 1}$ tal que $A\vec{c} \leq \vec{b}$?

Demuestre que INTEGERPROGRAMMING es NP-completo.

Solución: Es claro que INTEGERPROGRAMMING está en NP, pues podemos adivinar $\vec{c} \in \mathbb{Z}^{m \times 1}$ (que es de tamaño polinomial respecto a la entrada) y verificar en tiempo polinomial si se cumplen todas las desigualdades. Para probar que es NP-hard, haremos una reducción desde CNF-SAT.

Consideremos una fórmula ϕ en CNF con variables $p_1, \dots, p_n$. Por cada variable p_i vamos a considerar una variable x_i , y agregaremos al sistema las desigualdades $x_i \geq 0$ y $x_i \leq 1$ para que las soluciones al sistema que construiremos modelen valuaciones de la fórmula ϕ . Dado un literal ℓ , definimos

$$f(\ell) = \begin{cases} x_i & \text{si } \ell = p_i \\ 1 - x_i & \text{si } \ell = \neg p_i. \end{cases}$$

Por cada cláusula $\ell_{i,1} \vee \ell_{i,2} \vee \ell_{i,3}$ de ϕ agregamos al sistema la desigualdad (lineal)

$$f(\ell_{i,1}) + f(\ell_{i,2}) + f(\ell_{i,3}) \geq 1,$$

que corresponde a la condición de que alguno de los literales sea verdadero, es decir, que la cláusula sea satisfecha. Por último, notemos que podemos dejar el sistema de desigualdades en la forma requerida para las entradas de INTEGERPROGRAMMING dejando las constantes a un solo lado y posiblemente multiplicando por -1 cuando necesitemos invertir la dirección. Notemos que esta es una reducción de tiempo polinomial. $\square$

Ejemplo: Consideremos ϕ como $(p_1 \vee \neg p_2 \vee \neg p_3) \wedge (\neg p_1 \vee p_4) \wedge (p_2 \vee \neg p_3 \vee p_4 \vee p_5)$. Para cada $i \in \{1, 2, 3, 4, 5\}$ agregamos las desigualdades $0 \leq x_i \leq 1$, que equivalen a $x_i \leq 1$ y a $-x_i \leq 0$. Además, para las cláusulas agregamos las siguientes desigualdades:

- $x_1 + (1 - x_2) + (1 - x_3) \geq 1$, que equivale a $-x_1 + x_2 + x_3 \leq 1$.
- $(1 - x_1) + x_4 \geq 1$, que equivale a $x_1 - x_4 \leq 0$.
- $x_2 + (1 - x_3) + x_4 + x_5 \geq 1$, que equivale a $-x_2 + x_3 - x_4 - x_5 \leq 0$.

Por lo tanto, el sistema de programación entera que resulta de la reducción es el siguiente:

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & -1 \\ -1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & -1 & 0 \\ 0 & -1 & 1 & -1 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}.$$

Problema 3. Una fórmula de Horn es una fórmula proposicional en CNF en la que cada cláusula tiene a lo más un literal positivo. Demuestre que satisfacibilidad de una fórmula de Horn es un problema en P.

Solución: Dada una cláusula de Horn con un literal positivo, diremos que su “cabeza” es el literal positivo, y que los demás literales forman su “cuerpo”. Notemos que si en una fórmula de Horn todas las cláusulas tienen cuerpo no vacío, entonces podemos satisfacerla con la valuación que hace a todas las variables falsas. Por otro lado, una cláusula de Horn con cabeza pero sin cuerpo, es decir, que consiste de una sola variable, solo puede ser satisfecha si hacemos a esa variable verdadera. Basaremos el algoritmo de tiempo polinomial en estas dos observaciones:

```

Entrada: Una fórmula de Horn  $\phi$  con variables  $x_1, \dots, x_n$ ,
1: inicializar una lista clauses con todas las cláusulas de  $\phi$ ;
2: inicializar una  $n$ -tupla  $\sigma = (0, 0, \dots, 0)$ ;
3: inicializar un booleano terminado como False;
4: while not terminado do
5:   for  $C \in \text{clauses}$  do
6:     if  $C = x_i$  para algún  $i \in \{1, \dots, n\}$  then
7:       cambiar la  $i$ -ésima coordenada de  $\sigma$  a un 1;
8:     for  $C' \in \text{clauses} \setminus \{C\}$  do
9:       if  $C' = \neg x_i$  then
10:        retornar False;
11:      else if  $x_i$  es la cabeza de  $C'$  then
12:        quitar  $C'$  de la lista clauses;
13:      else if  $\neg x_i$  aparece en el cuerpo de  $C'$  then
14:        borrar  $\neg x_i$  del cuerpo de  $C'$ ;
15:      end if
16:    end for
17:    volver a la línea 4;
18:  end if
19: end for
20:  cambiar terminado a True;
21: end while
22: retornar True.

```

Si el algoritmo retorna **True**, la tupla σ será una valuación que satisface ϕ . Notemos que solo cambiamos un 0 por un 1 en σ cuando estamos obligados a hacerlo, y este cambio no será revertido más adelante. El bucle entre las líneas 8 y 16 corresponde a la actualización de conocimiento que realizamos una vez que hemos fijado un 1 en σ . Si entramos a la línea 10 es porque no había forma de satisfacer la fórmula, así que debemos retornar **False**. En la línea 12 podemos eliminar la cláusula C' porque ya sabemos que la valuación σ que estamos construyendo va a satisfacerla. Por último, en la línea 14 podemos eliminar $\neg x_i$ del cuerpo de C' porque sabemos que ese literal no ayudará a satisfacer dicha cláusula.

La razón por la que este algoritmo es de tiempo polinomial es que en cada iteración reducimos el tamaño de la fórmula, por lo que el número de iteraciones del bucle principal (entre las líneas 4 y 21) será lineal en el tamaño de la fórmula. Por otra parte, el número de iteraciones de los bucles interiores está acotado por el número de cláusulas de la fórmula original.

□